The Store — Arquitectura del POC: Terraform → Docker → kind (Kubernetes 1.35) → microservicios

Host: notebook Linux

Ubuntu 24.04 LTS · kernel 7.0 · Docker 29.7
Terraform 1.16 · kubectl 1.37 · 8 vCPU · 15 GB RAM

LAN / Internet (WiFi 802.11ax)

`wlp0s20f3 192.168.1.50/24`
`GW 192.168.1.1 → ISP → Internet`
(pull de imágenes públicas, charts de Helm)

Estación de trabajo (Terraform)

terraform apply · capas 1 → 2 → 3
provider docker: unix:///var/run/docker.sock
provider kind: librería kind v0.31 (sin binario)
providers helm/kubernetes: HTTPS 127.0.0.1:6443
estado local (3 tfstate) · secretos: random_password

Cliente (navegador / curl / tests)

`http(s)://the-store.localhost`
`http://dev.the-store.localhost`
*.localhost → 127.0.0.1 (RFC 6761, systemd-resolved)
HTTP/1.1 :80 · TLS 1.3 :443 (certificado autofirmado)

Interfaces y puertos publicados

`br-kind 172.30.0.1/24 (gateway)`
`0.0.0.0:80,443 → 172.30.0.2:80,443`
`127.0.0.1:6443 → 172.30.0.2:6443`
`127.0.0.1:5001 → 172.30.0.100:5000`
iptables: DNAT (docker-proxy) a contenedores
y SNAT MASQUERADE 172.30.0.0/24 → 192.168.1.50

Plan de direccionamiento (RFC 1918)

`LAN host 192.168.1.0/24`
`Nodos (docker) 172.30.0.0/24`
`Pods 10.244.0.0/16 (/24 x nodo)`
`Services 10.96.0.0/16 (ClusterIP)`
bloques disjuntos entre sí y con la LAN del host

Referencias

- tráfico norte-sur (usuario → aplicación)
- tráfico este-oeste (pods, HTTP 8080)
- plano de control / Terraform / pull
- DMZ: único punto de entrada (ingress)
- namespace the-store-prod (NetworkPolicies)
- namespace the-store-dev (in-memory)
- capa de datos (StatefulSet + PVC 1 Gi)

SO nodos: Debian 12 (kindest/node) · SO pods app: Amazon Linux 2023 · datos: Debian/Alpine oficiales

Red Docker «kind» — bridge L2 (un dominio de broadcast) + NAT

`172.30.0.0/24 · gateway 172.30.0.1 · DNS interno de Docker (nombres de contenedor) · un par veth por contenedor`

kind-registry (registry:3)

`172.30.0.100 · TCP 5000 (HTTP v2)`
push desde el host: localhost:5001
pull desde nodos: alias de containerd
`localhost:5001→kind-registry:5000`

the-store-control-plane (nodo edge) — 172.30.0.2 — pods 10.244.0.0/24

Debian 12 · containerd 2.2 · kubelet v1.35.0 · taint control-plane (no corre cargas de aplicación)

kube-apiserver
`HTTPS 6443 (mTLS)`
etcd 2379 · scheduler · ctrl-mgr

CoreDNS ×2
`10.96.0.10 · 53 UDP/TCP`
*.svc.cluster.local

kindnet + kube-proxy
CNI: ruta estática /24 por nodo
kube-proxy: DNAT iptables (L4)

metrics-server · local-path
Metrics API (HPA, kubectl top)
PV en /var/local-path-provisioner

DMZ · ingress-nginx (reverse proxy L7, hostPort 80/443, nodeSelector ingress-ready=true)
Enruta por cabecera Host: the-store.localhost → svc ui (prod) · dev.the-store.localhost → svc ui (dev)
Internet→DMZ: sólo 80/443 · DMZ→interno: sólo ui:8080 (NetworkPolicy) · Internet→interno: bloqueado (ningún pod publica puertos)
TLS 1.3 con certificado autofirmado (Secret kubernetes.io/tls generado por Terraform) · HTTP sin redirección forzada

Terraform → API server 127.0.0.1:6443
1. DNAT Docker (docker-proxy): host 0.0.0.0:80/443 → 172.30.0.2:80/443

pull de imágenes localhost:5001/the-store-*:<hash> (ambos workers)

the-store-worker — 172.30.0.3 — pods 10.244.2.0/24

Debian 12 · containerd 2.2 · kubelet v1.35.0 · label tier=workload

the-store-worker2 — 172.30.0.4 — pods 10.244.1.0/24

Debian 12 · containerd 2.2 · kubelet v1.35.0 · label tier=workload

2. ingress-nginx: Host the-store.localhost → svc ui (ClusterIP :80) → DNAT kube-proxy → pod ui :8080

namespace the-store-prod — PSA baseline — NetworkPolicies: default-deny (ingress+egress) + flujos explícitos + DNS

ui ×2 (+HPA 2–4)
Java 21 / Spring Boot · :8080
readiness /actuator/health
PDB minAvailable=1

catalog ×2
Go / Gin · :8080
PERSISTENCE=mysql
PDB minAvailable=1

carts ×1
Java 21 / Spring Boot · :8080
PERSISTENCE=dynamodb
AWS SDK → DynamoDB Local

orders ×1
Java 21 / Spring Boot · :8080
persistence=postgres
messaging=rabbitmq

checkout ×1
Node 20 / NestJS · :8080
persistence=redis
ENDPOINTS_ORDERS=http://orders

ui → catalog / carts / orders / checkout (HTTP 8080)
checkout → orders (HTTP 8080)

catalog-db
`mariadb:11.4 · 3306/TCP`
protocolo MySQL · catalogdb
Secret: MARIADB_PASSWORD
PVC 1 Gi en /var/lib/mysql
sólo alcanzable desde catalog

carts-db
`dynamodb-local · 8000/HTTP`
API DynamoDB (JSON sobre HTTP)
-sharedDb -dbPath (PVC 1 Gi)
credenciales dummy del SDK
sólo alcanzable desde carts

orders-db
`postgres:17-alpine · 5432`
protocolo PostgreSQL · orders
Secret: POSTGRES_PASSWORD
PVC 1 Gi (PGDATA)
sólo alcanzable desde orders

orders-mq
`rabbitmq:3.13 · 5672/AMQP`
consola HTTP 15672 (ClusterIP)
Secret: RABBITMQ_DEFAULT_PASS
PVC 1 Gi
sólo alcanzable desde orders

checkout-redis
`redis:7-alpine · 6379/RESP`
appendonly (AOF) en PVC 1 Gi
sin autenticación: aislado por NetworkPolicy
sólo alcanzable desde checkout

Pods repartidos entre ambos workers por el scheduler · tráfico entre nodos: 10.244.1.0/24 via 172.30.0.4 y 10.244.2.0/24 via 172.30.0.3 (rutas estáticas de kindnet, sin túnel/overlay)

namespace the-store-dev — mismo código Terraform, tfvars de dev: proveedores in-memory, 1 réplica, sin NetworkPolicies, sin TLS

ui ×1
endpoints → los 4 backends
Host: dev.the-store.localhost

catalog ×1
in-memory

carts ×1
in-memory

orders ×1
in-memory (datos y mensajería)

checkout ×1
in-memory → http://orders

Services ClusterIP 10.96.0.0/16 (puerto 80 → targetPort 8080) · DNS: <svc>.the-store-dev.svc.cluster.local · Ingress: dev.the-store.localhost → svc ui
Entornos: terraform workspace select dev|prod + -var-file=environments/<env>.tfvars sobre el mismo código de la capa 3
Egreso a Internet: pod 10.244.x.y → nodo 172.30.0.x → SNAT MASQUERADE del bridge → 192.168.1.50 → GW 192.168.1.1 (NAT del ISP): «doble/triple NAT»